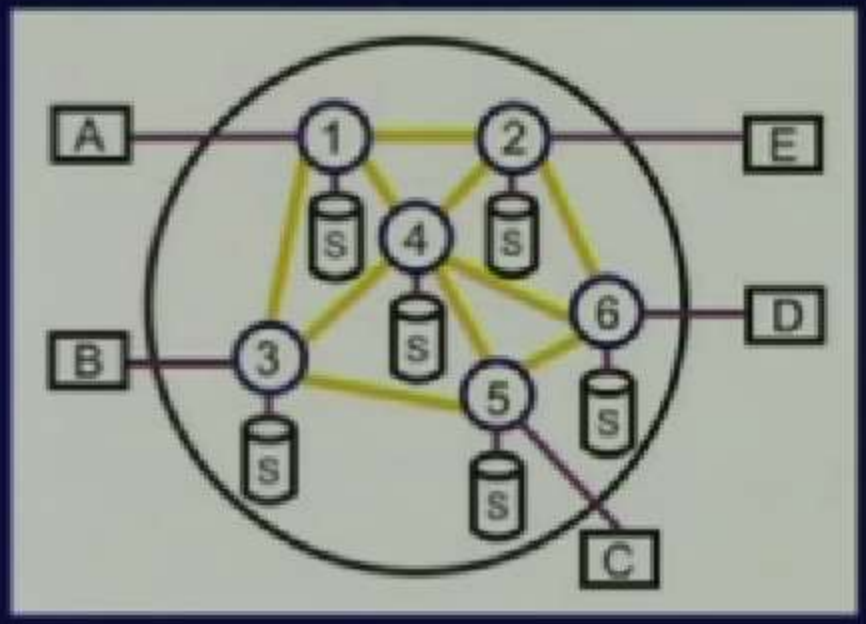
- Why congestion?
- Common causes of congestion
- Effects of Congestion
- Open loop congestion control techniques:
  - The leaky bucket algorithm
  - The token bucket algorithm
- Close loop congestion control techniques:
  - Admission Control
  - Weighted fair queuing
  - Resource reservation
  - Choke packets
  - Load shedding
- Congestion versus Flow control

Why Congestion?



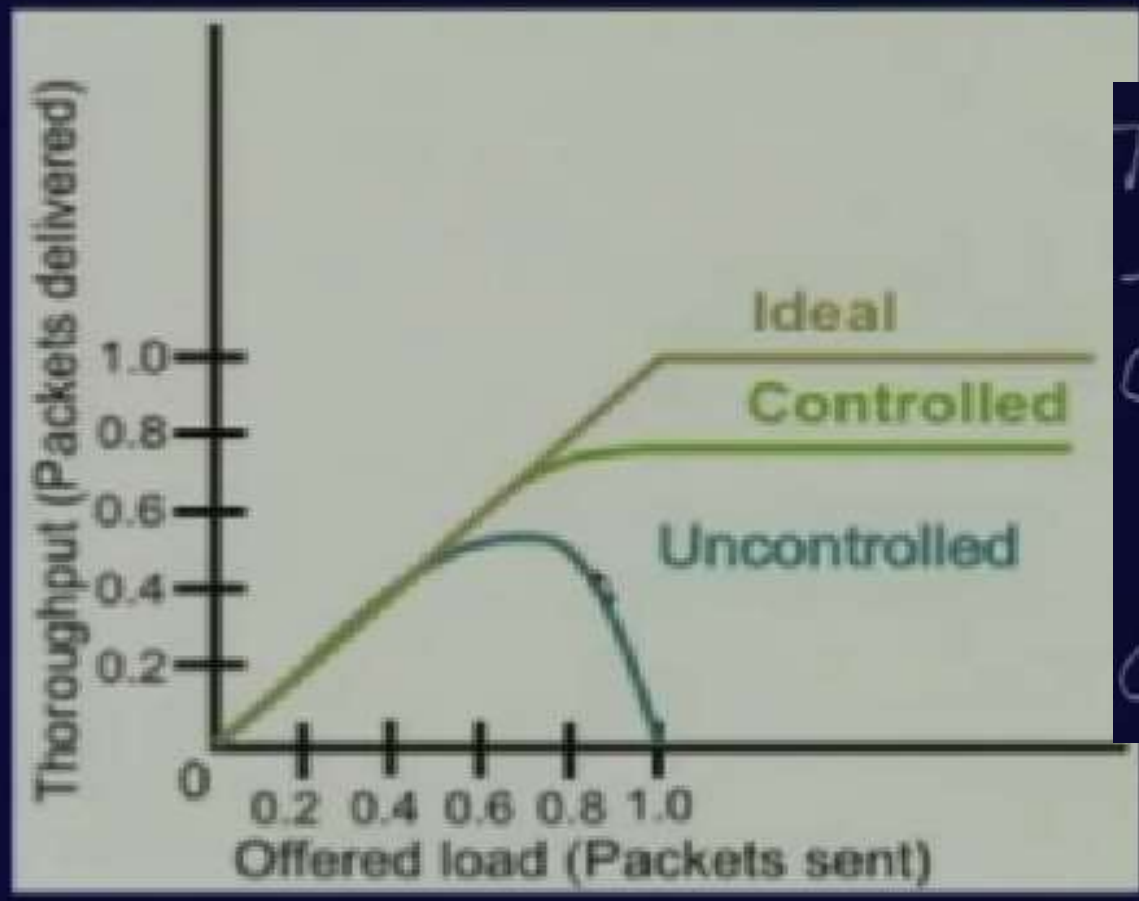
Common Causes

- As packets arrive at a node, they are stored in an input buffer. If packets arrive too fast, an incoming packet may find that there is no available buffer space.
- Even very large buffer cannot prevent congestion, because of delay, timeout and retransmissions.
- Slow processors may lead to congestion.
- Low bandwidth lines may also lead to congestion.

Congestion Defined

- If packet arrival rate exceeds the packet transmission rate, the queue size grows without bound.
- Delay in delivery of packets leads to retransmissions.
- Thumb rule: When the line for which packets are queuing becomes more than 80% utilized, the queue length grows alarmingly.
- When too many packets arrive a part of the packet-switched network, the performance degrades. This is is known as congestion.

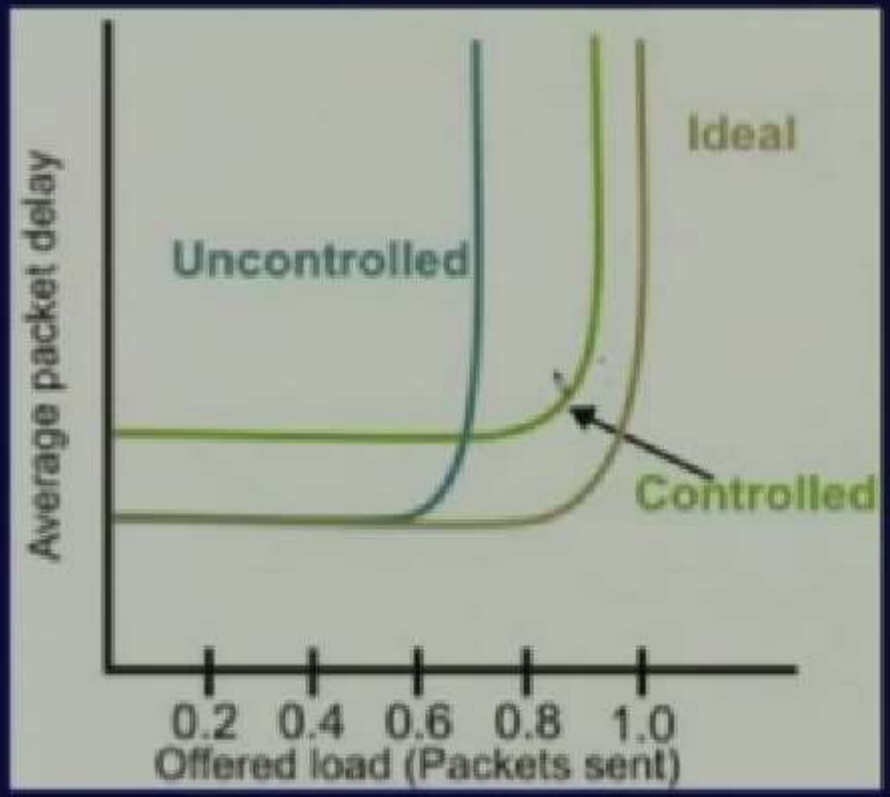
Effect of congestion



Thrashing

Congestion can be controlled

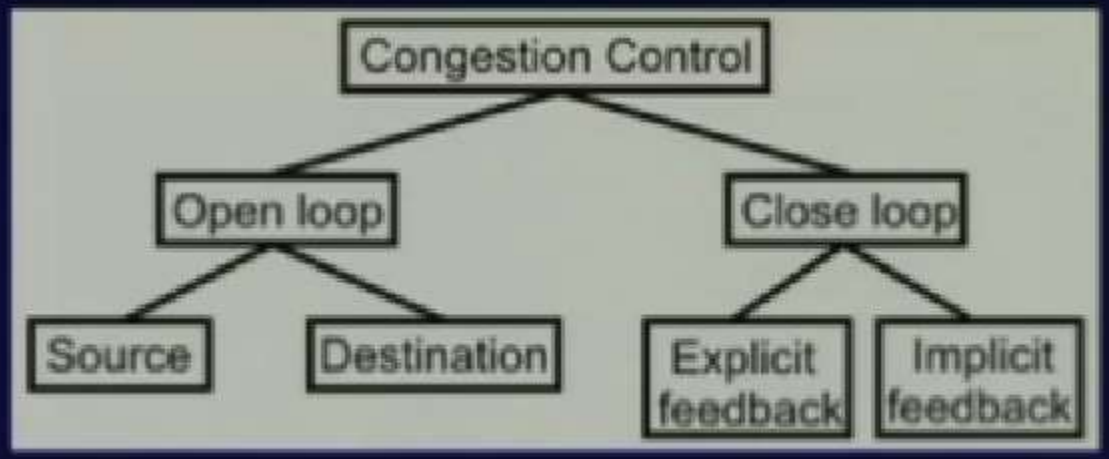
Effect of congestion



Delay
Propagation time +
Transmission time

Principles of Congestion Control

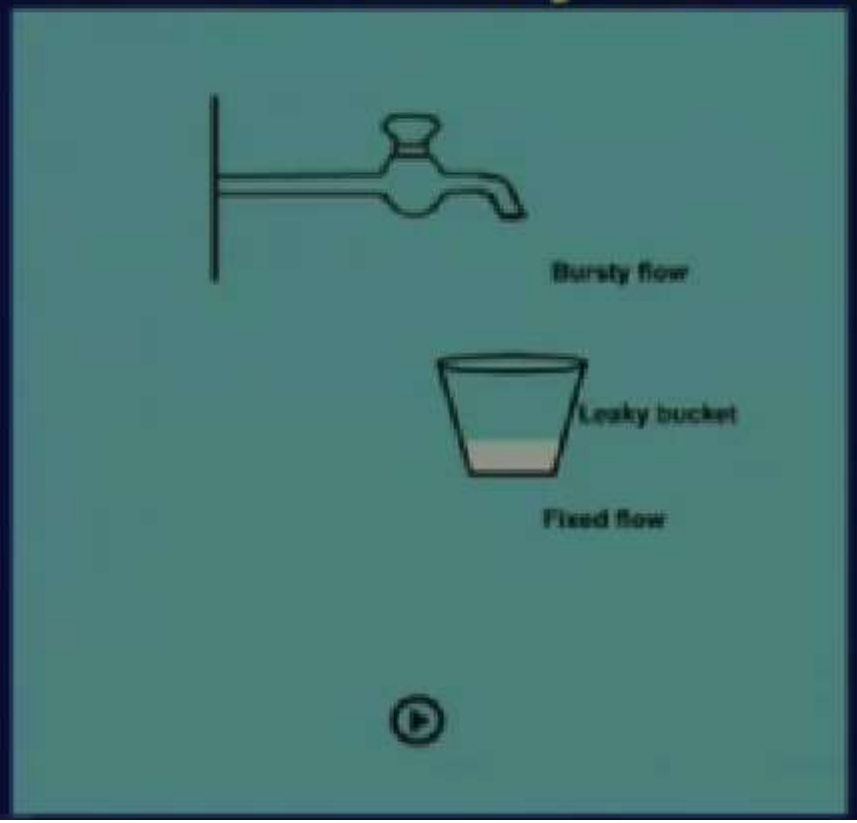
- **Congestion control** refers to the mechanisms and techniques used to control congestion and keep the traffic below the capacity of the network.

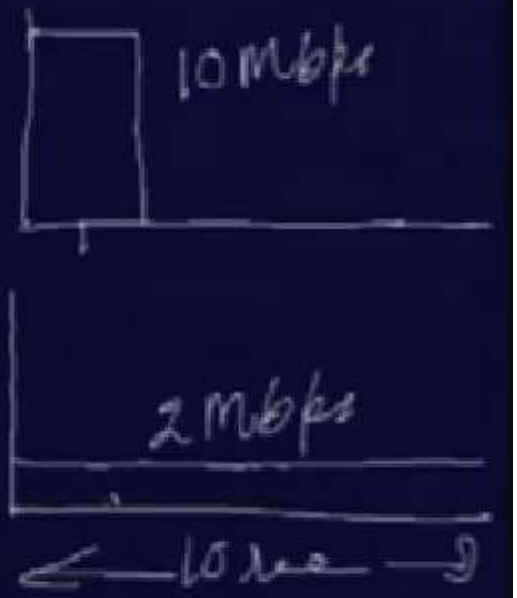


Open Loop Congestion Control

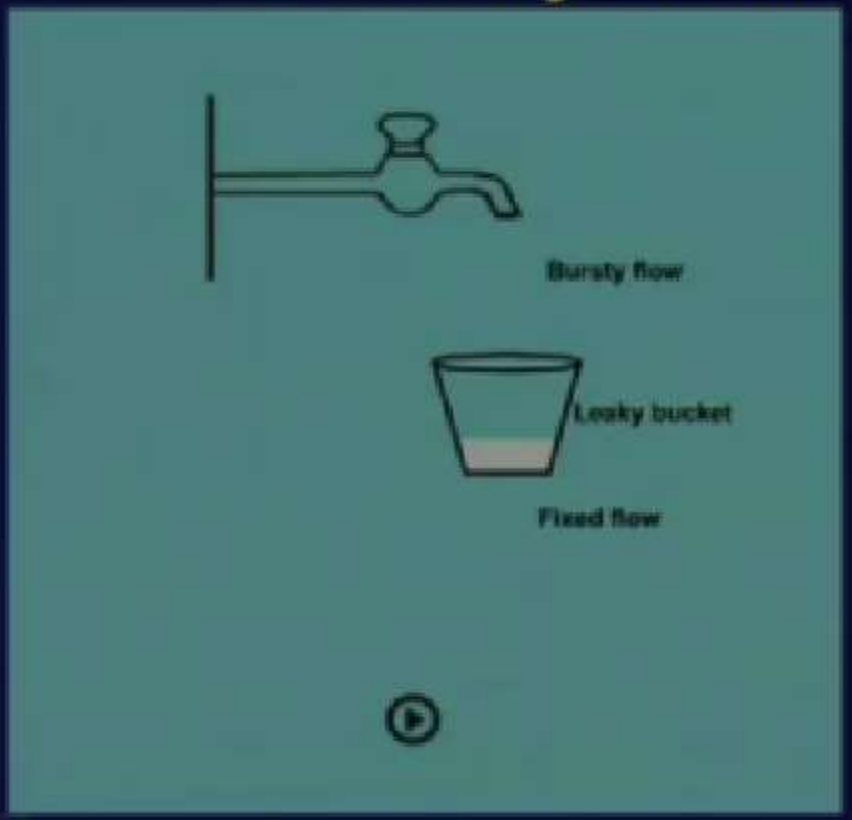
- The basic objective is to prevent congestion by adopting suitable policies for:
  - Flow control
  - Acknowledgement
  - Retransmission (Timeout interval)
  - Caching
  - Packet discard
  - Routing
- Traffic Shaping:
  - The leaky bucket algorithm
  - The token bucket algorithm

The Leaky Bucket Algorithm





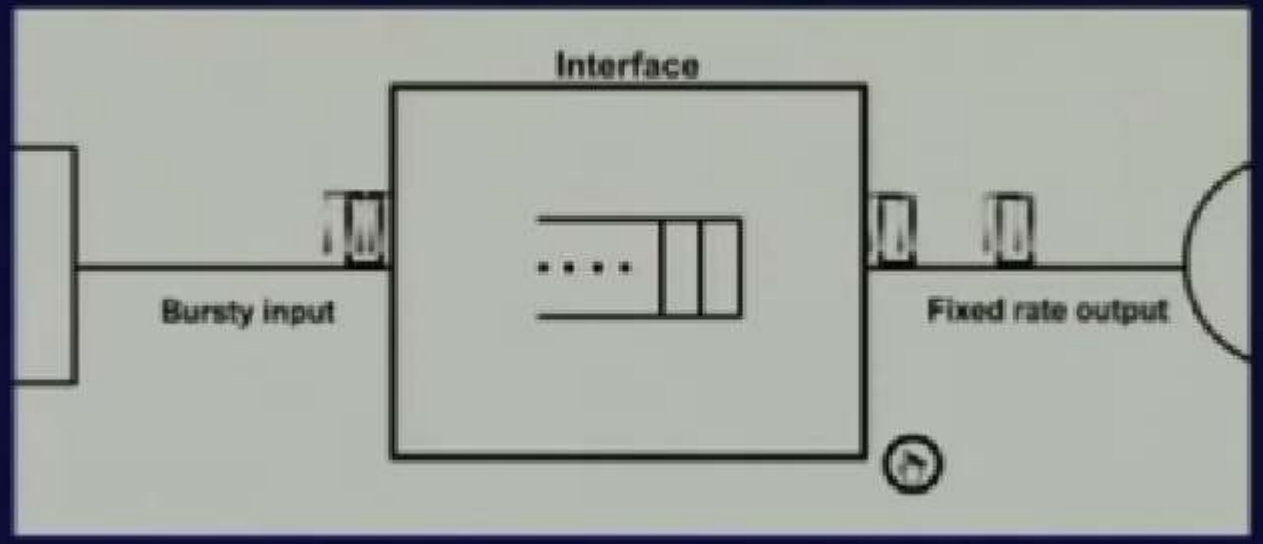
The Leaky Bucket Algorithm



It shapes bursty traffic into fixed rate traffic

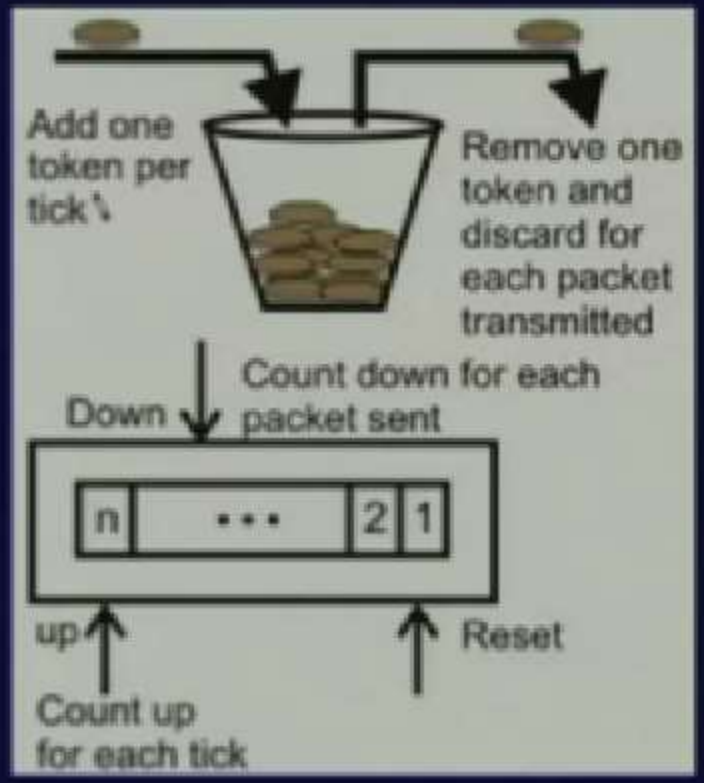
Packets are dropped when the bucket is full

The Leaky Bucket Algorithm



Limitation: Enforces rigid output pattern even when the traffic in the network is small

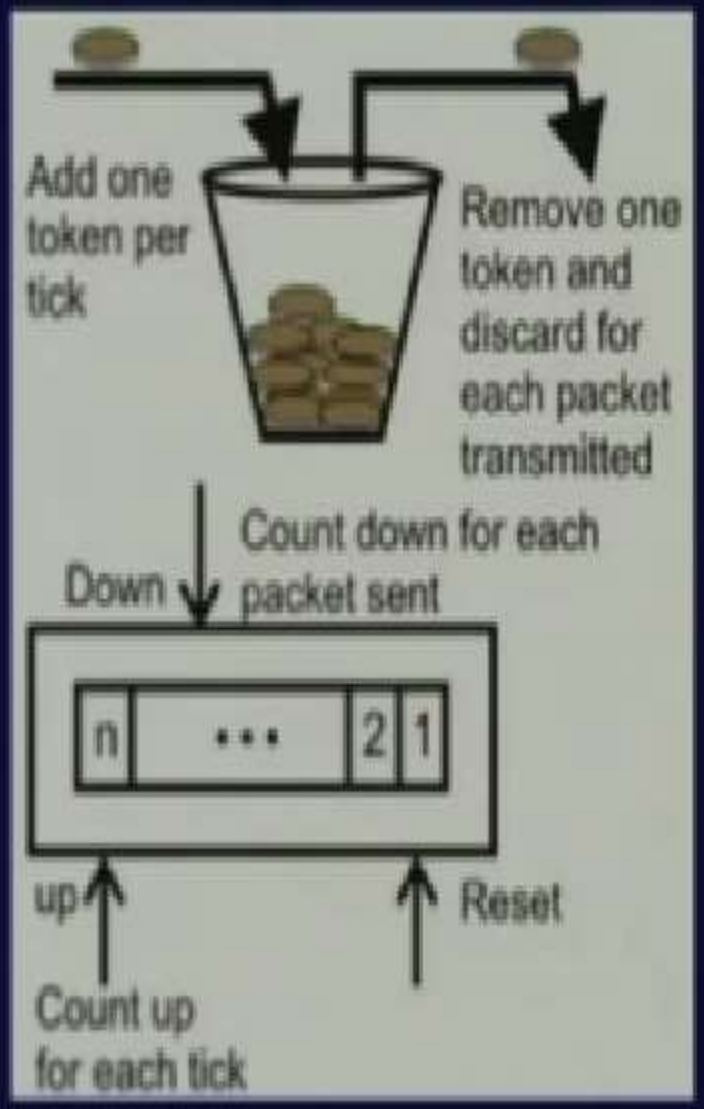
Token Bucket Algorithm



Per each tick

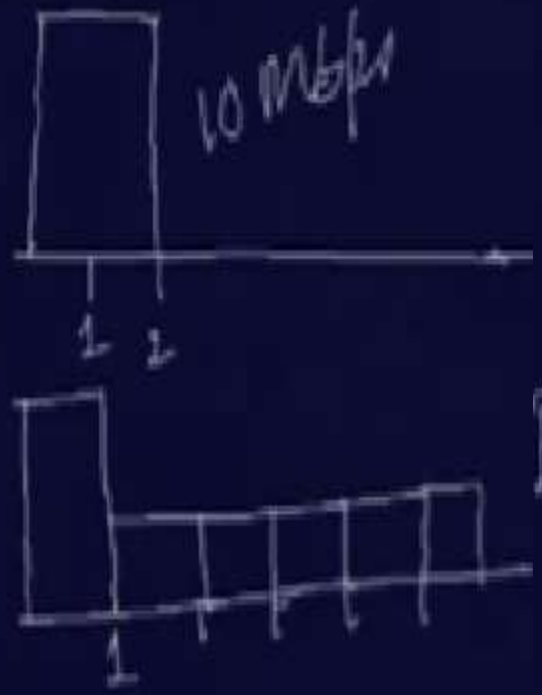
Δt

Token Bucket Algorithm



Per each tick

At



better Throughput

The token bucket algorithm saves up to a maximum of N tokens and allows a burstiness of N packets in the output stream, thereby giving faster response.

An important difference between two traffic shaping algorithms: token bucket throws away tokens when the bucket is full but never discards packets while leaky bucket discards packets when the bucket is full. Unlike leaky bucket, token bucket allows saving, up to maximum size of bucket n. This means that bursts of up to n packets can be sent at once, giving faster response to sudden bursts of input. Leaky bucket forces bursty traffic to smooth out, token bucket permits burstiness but bounds it. Token bucket has no discard or priority policy. Token bucket when compared to leaky bucket, is easy to implement. Each flow needs just a counter to count tokens and a timer to determine when to add new tokens to the counter.

Flow Specification

- To make traffic shaping approaches successful, it is necessary to specify the requirement in a precise manner known as **Flow Specification.**

- Input characteristics:
  - Maximum packet size Kbits
  - Token bucket rate Mb/sec
  - Token bucket size GB
  - Maximum transmission rate

- Services desired
  - Reliability
  - Delay
  - Jitter Variation of Delay
  - Bandwidth

Close Loop Congestion Control

- Three basic steps:
  - Monitor the system for detection of congestion (when and where it occurs).
  - Pass the information to proper place for taking action to control congestion.
  - Adjust system parameters to get out of congestion
- Various approaches:
  - Admission Control
  - Choke packets
  - Weighted fair queuing
  - Load shedding
  - Resource reservation

Admission Control

- Used in Virtual Circuit Networks
- When congestion is detected no more virtual circuit is allowed to set up.
- Or virtual circuit is allowed to be set up around the congested subnets
- An agreement between a host and a subnet is negotiated when a virtual circuit is set up so that resources along the path are guaranteed throughout the session.
- This may lead to underutilization of the resources.

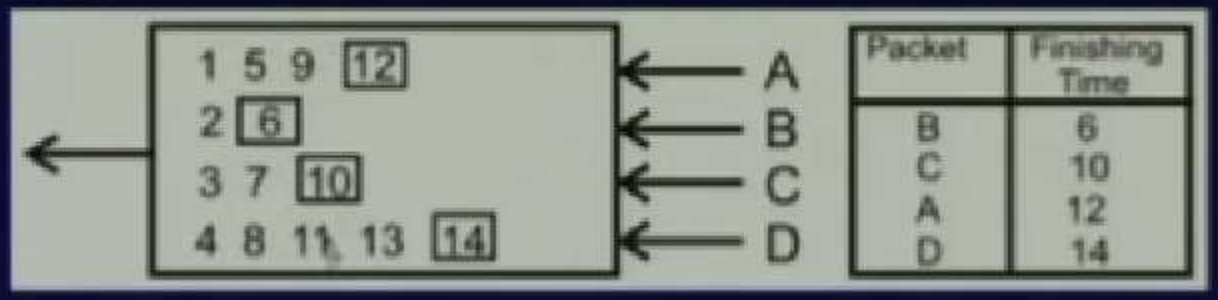
Choke Packets

- Can be used for both virtual circuit and datagram type of networks
- Each node monitors the utilization of the output lines.
- If the utilization goes beyond some threshold level, an output line enters a 'warning state'.
- If the output line of a newly arriving packet is 'warning state', a control packet (called choke packet) is sent from a congested node to the source station.
- The original packet is tagged so that it does not generate any more choke packet.
- After receiving a choke packet, the source station reduces traffic by certain percentage.

50%
25%
10%

Fair Queuing

- Choke packet approach has the drawback that the action taken by a source in voluntary.
- To overcome this problem, fair queuing algorithm is used, where queues are scanned in a round-robin manner.
- Packets are sorted in order of their finishing time and sent in that order.



Packet	Finishing Time
B	6
C	10
A	12
D	14

Weighted Fair Queuing

- Instead of giving same priority to all the the hosts, higher priority can be given to some hosts, such as server, leading to the modified algorithm known as **weighted fair queuing**.

Data Recent

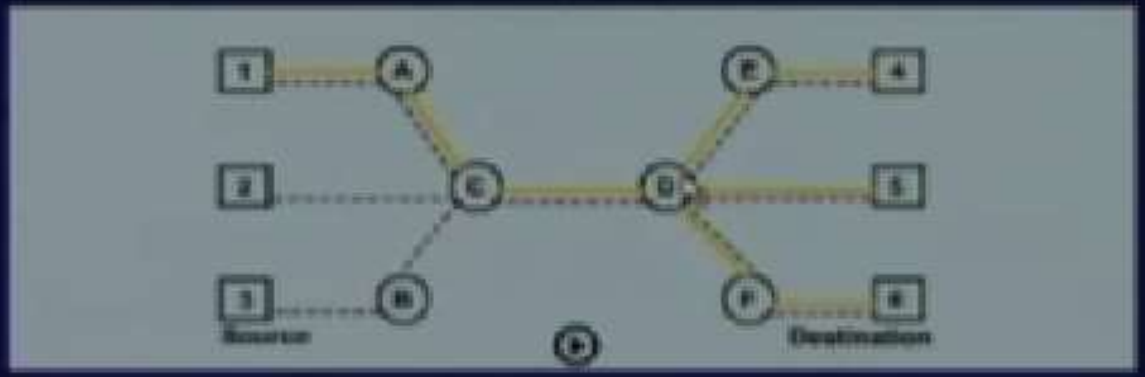
Video Old

Load Shedding

- When other methods fail, the nodes can resort to a heavy artillery: Load Shedding.
- When the nodes are not able to handle the packets, packets are discarded.
- A node drowning in packets, one approach is to discard packets at random.
- In many situations, some packets are more important than others.
- To implement this intelligent discard policy, the applications must mark the packets with priority classes.
- Simulation results show that it is better to start discarding packets early, rather than wait until the network is completely clogged up

RSVP – Resource reSerVation Protocol

- Multicast flows from multiple sources to multiple destinations
- Uses multicast routing using spanning trees.
- Each group is assigned a group address, which is used to send packets to that group
- The multicast routing algorithm builds a spanning tree covering all group members.
- To eliminate congestion any of the receivers can send reservation message up the tree to the sender.



Congestion Versus Flow Control

- **Congestion control:**
- Congestion control is a global issue. It is a joint responsibility of the users and the network.
- It occurs because of the combined behavior of the stations, switches, routing policies, etc
- Its job is to ensure that the subnet is able to carry the offered load.

- **Flow control:**
- Flow control is a local issue between a sender-receiver pair.
- Its primary function is to ensure that a fast sender does not overwhelm a slow receiver.

1. In what way random routing is superior to flooding?

Ans: The random routing significantly reduces the number of copies of a single packet to be transmitted through the network compared to flooding, but has the same level of simplicity and robustness of flooding .

2. Why adaptive routing is preferred over static routing?

Ans: Despite several drawbacks, adaptive routing is widely used because they Improve performance, works under changing conditions on the network and can aid in congestion control.

3. What are the limitations of adaptive outing?

Ans: Routing decision is more complex In adaptive routing, which imposes more processing burden on the switching nodes.

- Traffic overhead is more in adaptive routing
- If adaptive routing algorithm reacts too quickly to changing network state, it may lead to congestion-producing oscillation in the network.

4. Compare and contrast distance vector routing with link state routing.

- Distance vector:
- Knowledge about the entire network
- Routing only to neighbors
- Information sharing at regular interval

- Link state:
- Knowledge about the neighborhood
- Routing to all
- Information sharing at regular interval
- Converges quickly
- Requires more CPU power and memory
- more scalable